Examen — Codage et traitement du signal

Détection de notes et accordeur chromatique en temps réel

basés sur la Transformée de Fourier Discrète

RALISON Andriniaina Stephano Michel
L2 Informatique

Date de rendu : 10 novembre 2026

1. Objectif et architecture

Le projet consiste à capter un son (voix, guitare, etc.) par le microphone, à en extraire la **fréquence fondamentale** grâce à la Transformée de Fourier Discrète (TFD), puis à afficher la note la plus proche de la gamme tempérée ainsi que l'écart d'accordage en cents, à la manière d'un accordeur chromatique. Le traitement est réalisé en temps réel, trame après trame, en Python (NumPy, Matplotlib, sounddevice).

Architecture du code

```
accordeur/dft.py        TFD directe O(N^2) et FFT radix-2 (Cooley-Tukey) écrites à la main
accordeur/detection.py  fenêtrage, zero-padding, HPS, interpolation, extraction de f0
accordeur/notes.py      fréquence -> note (nom, octave, écart en cents)
accordeur/audio.py      capture micro (tampon circulaire), source simulée, lecture WAV
main.py                 interface temps réel (spectre + jauge), modes console / fichier
tests/                  tests unitaires (pytest)
generer_rapport.py      figures et rapport PDF
```

Chaîne de traitement : **micro** → trame de 8192 échantillons (Fe = 44100 Hz, soit 186 ms) → suppression de la composante continue → fenêtre de Hann → zero-padding (x4, soit 32768 points) → FFT → HPS → interpolation parabolique → conversion en note → lissage par médiane glissante → affichage.

2. Transformée de Fourier Discrète

Pour un signal échantillonné x[n], n = 0..N-1, la TFD est définie par : $\mathbf{X[k] = \Sigma\, x[n]\cdot e^{-2i\pi kn/N}}$, k = 0..N-1. Le coefficient X[k] mesure la présence de la fréquence f_k = k·Fe/N dans le signal. Pour un signal réel, le spectre est symétrique ; on ne conserve que les fréquences de 0 à Fe/2 (théorème de Shannon-Nyquist).
La TFD directe coûte O(N²) opérations. L'algorithme de **FFT de Cooley-Tukey** (radix-2) exploite la décomposition récursive en indices pairs et impairs pour atteindre O(N log N) : pour N = 32768, cela représente environ 500 000 opérations de papillons au lieu d'un milliard, ce qui rend le temps réel possible. Les deux versions sont implémentées dans `dft.py` ; leur écart maximal sur un signal aléatoire de 512 points est de 7.9e-12 (erreurs d'arrondi uniquement).

Résolution fréquentielle et compromis

La résolution naturelle de la TFD est Fe/N = 44100/8192 ≈ 5,4 Hz. Elle est insuffisante pour distinguer deux notes graves (le Mi2 à 82,41 Hz et le Fa2 à 87,31 Hz sont séparés de moins de 5 Hz) avec la précision d'un accordeur (quelques cents). Trois techniques permettent de l'améliorer sans allonger la trame (donc sans perdre en réactivité) :
• **Fenêtre de Hann** : un signal n'est jamais périodique sur exactement N échantillons, ce qui provoque une *fuite spectrale* (figure 3). La fenêtre de Hann réduit fortement les lobes secondaires.
• **Zero-padding** : ajouter des zéros avant la FFT interpole le spectre (pas fréquentiel réduit à ≈ 1,35 Hz) sans ajouter d'information, mais facilite la localisation du pic.
• **Interpolation parabolique** : on ajuste une parabole sur le logarithme du spectre autour du pic (3 points) ; l'abscisse du sommet donne la fréquence avec une précision très inférieure au pas de la TFD.

3. Extraction de la fréquence fondamentale

La fréquence fondamentale f0 est la plus basse fréquence du son périodique ; l'oreille perçoit la hauteur de la note correspondante. Dans un instrument réel, le spectre contient f0 et ses **harmoniques** (2·f0, 3·f0, …). Le pic le plus haut n'est pas toujours la fondamentale (figure 2 : à 220 Hz, le deuxième harmonique domine), ce qui conduirait à une **erreur d'octave** si l'on prenait simplement le maximum du spectre.

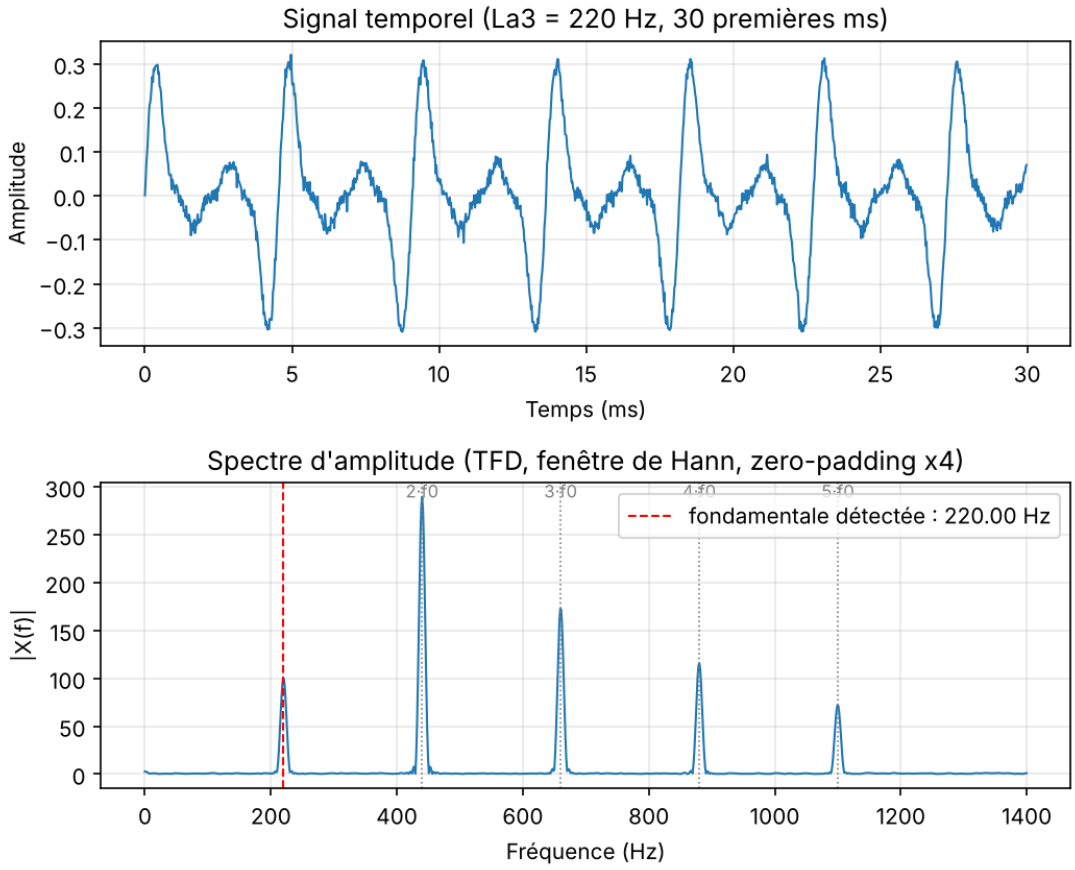


Figure 1 — Signal temporel et diagramme du spectre fréquentiel d'un La3 (220 Hz) avec harmoniques.

Algorithme utilisé (`detection.py`)

1. **Seuil de silence** : si la valeur efficace (RMS) de la trame est inférieure à 0,01, aucune note n'est affichée.
2. **Prétraitement** : suppression de la moyenne, multiplication par la fenêtre de Hann, zero-padding à 4N points.
3. **TFD (FFT)** et calcul du module |X[k]|.
4. **Produit spectral harmonique (HPS)** : HPS[k] = $\Pi_{h=1..5}$ |X[h·k]|. Si f0 est la fondamentale, ses harmoniques 2f0, 3f0, … alignent leurs pics sur h·k : le produit présente alors un maximum net en k0 (figure 2).
5. **Recherche du maximum** du HPS entre 50 Hz et 2000 Hz, puis vérification de la sous-octave pour corriger une éventuelle erreur d'octave.
6. **Interpolation parabolique** du pic sur le spectre en échelle logarithmique : k* = k + ½·(a − c)/(a − 2b + c), où a, b, c sont les valeurs en k−1, k, k+1. La fréquence est f0 = k*·Fe/N_fft.

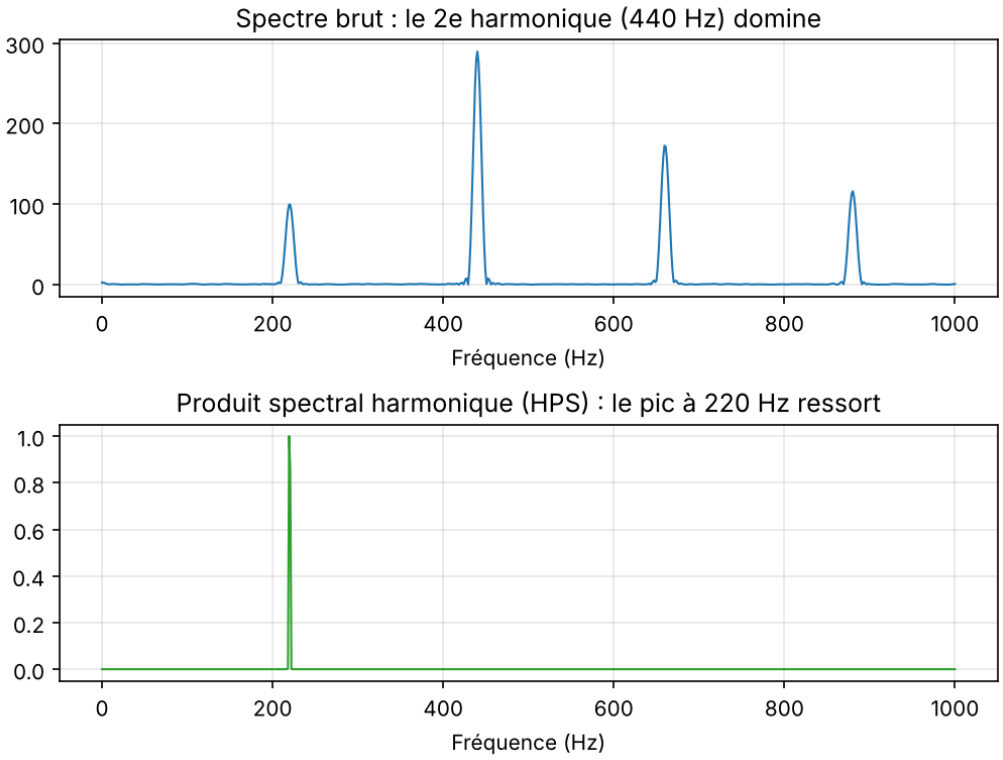


Figure 2 — Spectre brut et produit spectral harmonique pour un La3.

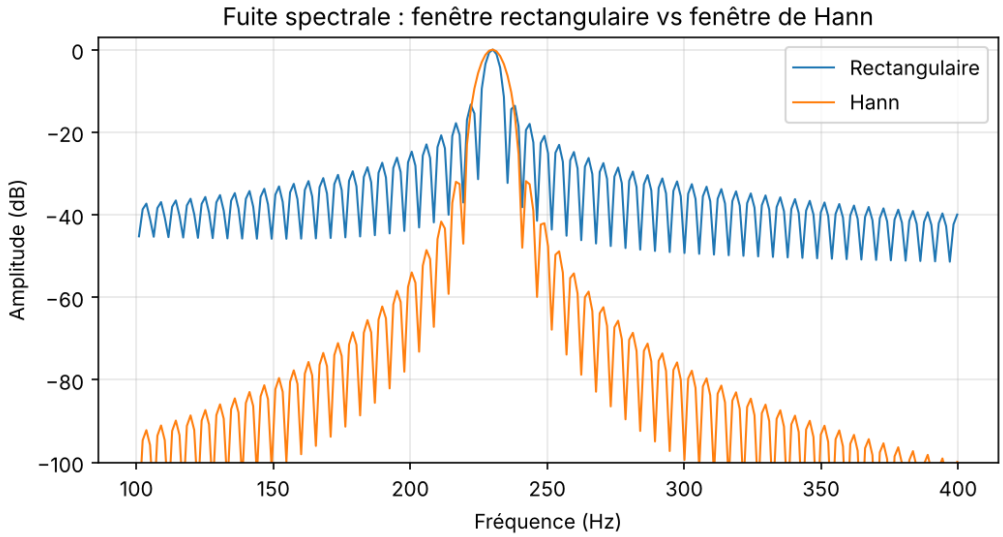


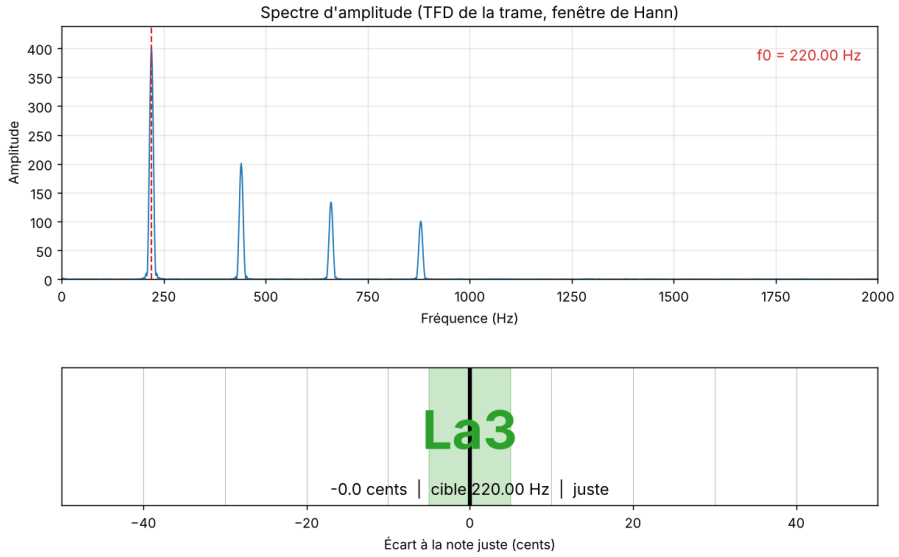
Figure 3 — Effet du fenêtrage sur la fuite spectrale (sinusoïde de 230 Hz).

4. Conversion en note de musique

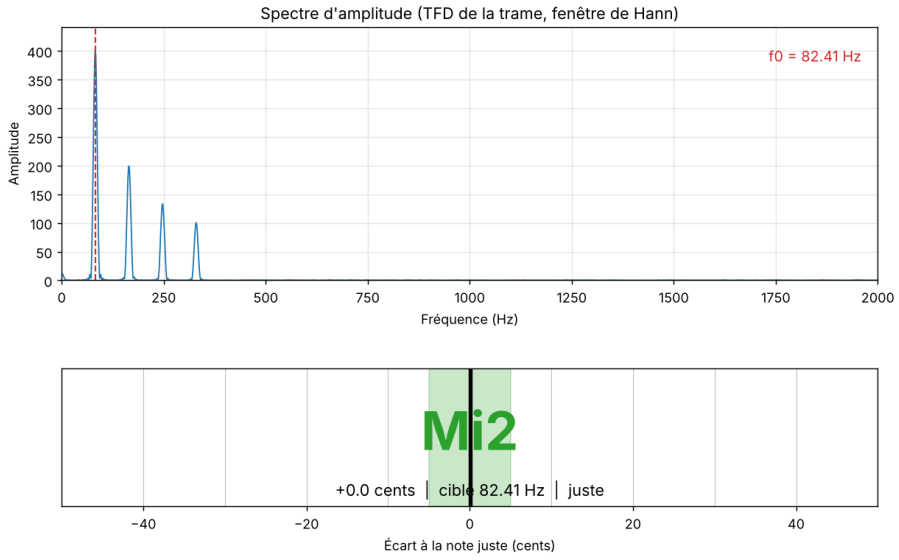
Dans la gamme tempérée, deux demi-tons consécutifs sont dans le rapport $2^{1/12}$. Avec La4 = 440 Hz, le numéro MIDI d'une fréquence f est $\mathbf{m = 69 + 12 \cdot \log_2(f/440)}$. La note la plus proche est obtenue en arrondissant m ; son nom est donné par m mod 12 (Do, Do#, Ré, ...) et son octave par $\lfloor m/12 \rfloor - 1$. L'écart d'accordage en **cents** (1 demi-ton = 100 cents) vaut $\mathbf{1200 \cdot \log_2(f \,/\, f_{théorique})}$. L'accordeur affiche la note en vert lorsque |écart| ≤ 5 cents.

5. Interface temps réel

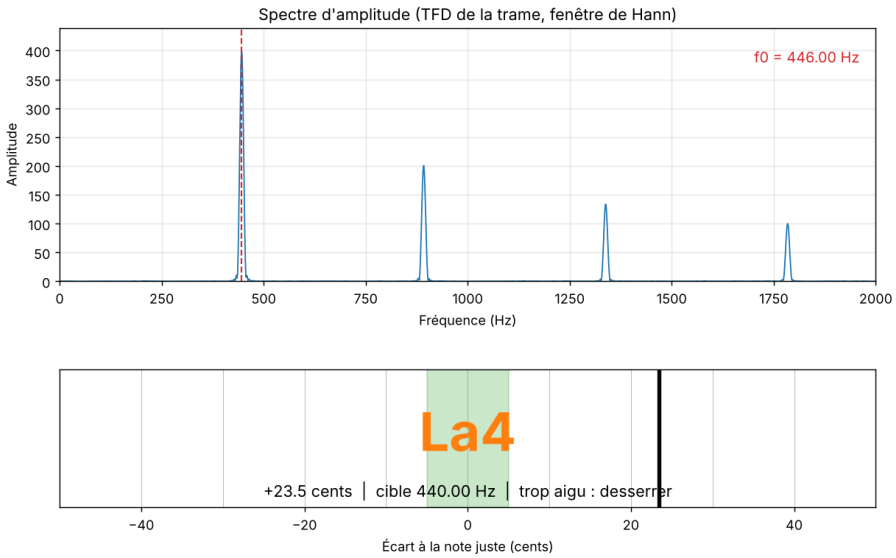
Le microphone est lu par un flux `sounddevice` (blocs de 1024 échantillons) qui alimente un tampon circulaire des 8192 derniers échantillons. Toutes les 80 ms, la dernière trame est analysée et l'interface Matplotlib est rafraîchie : spectre en haut, note détectée et jauge en cents en bas. Une médiane glissante sur les 5 dernières détections stabilise l'affichage. Les captures ci-dessous ont été obtenues avec l'application en mode source simulée (`python main.py --capture ... --demo`) : un instrument virtuel avec harmoniques et bruit, ce qui rend les mesures reproductibles.



Capture 1 — Détection d'un La3 (220 Hz) accordé.



Capture 2 — Détection d'un Mi2 (82,41 Hz), corde grave de guitare.



Capture 3 — La4 désaccordé (446 Hz, +23,5 cents) : « trop aigu ».

6. Validation

Des signaux synthétiques (fondamentale + 4 harmoniques, bruit gaussien) ont été analysés. L'erreur reste inférieure à quelques cents sur toute la tessiture d'une guitare et d'une voix, même lorsque la fondamentale est plus faible que le 2e harmonique (cas du La3 testé ici).

Note	f théorique (Hz)	f détectée (Hz)	Erreur (cents)
Mi2	82.41	82.41	-0.10
La2	110.00	110.00	+0.07
Ré3	146.83	146.84	+0.09
Sol3	196.00	195.98	-0.14
Si3	246.94	246.94	+0.01
Mi4	329.63	329.64	+0.04
La4	440.00	440.03	+0.11
Do5	523.25	523.22	-0.10
La5	880.00	879.98	-0.04

Tableau 1 — Précision de la détection (signaux synthétiques bruités).

Les 14 tests unitaires (`pytest`) vérifient : l'égalité entre la TFD directe, la FFT maison et `numpy.fft` ; le rejet des longueurs invalides ; l'erreur de moins de 3 cents sur 9 notes ; l'absence d'erreur d'octave avec fondamentale atténuée ; la gestion du silence ; la conversion en notes.

7. Limites et perspectives

• La résolution reste limitée pour les notes très graves (< 60 Hz) avec une trame de 186 ms ; une trame plus longue améliore la précision au prix de la réactivité.
• Les sons polyphoniques (accords) ne sont pas traités : HPS suppose une seule fondamentale.
• Des méthodes temporelles (autocorrélation, YIN) pourraient compléter l'approche spectrale.
• Le diapason (La4 = 440 Hz) est un paramètre modifiable dans `notes.py`.

8. Utilisation

```
pip install -r requirements.txt
python main.py                 # micro + interface graphique
python main.py --demo          # source simulée
python main.py --console       # affichage texte
python main.py --fichier x.wav # analyse d'un WAV
python -m pytest               # tests
```